

Lab 5

Lab Content:

1. Transient Response
2. Steady State Error
3. PID Controller
4. PID Tuning Tool
5. Implementing PID Control on Arduino
6. Root Locus
7. GUI and Connecting it to Arduino

1. PID Control Theory:

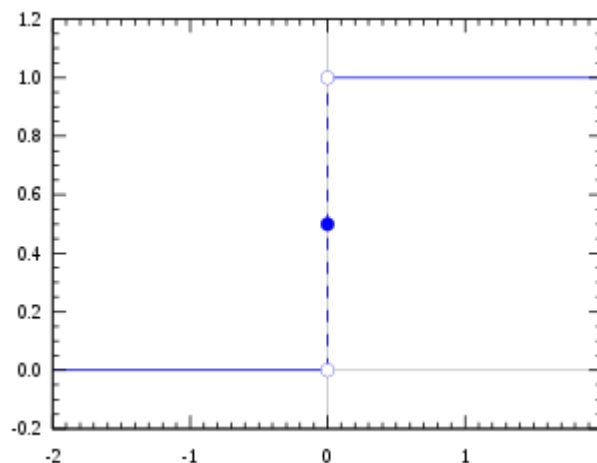
First of all, let us define some basic definitions that will be needed for deducing the formula of a digital PID controller which is to be used to produce a control action; that will compensate an error produced from some negative feedback.

- Step Response:

As predefined the unit step function is defined as per the following:

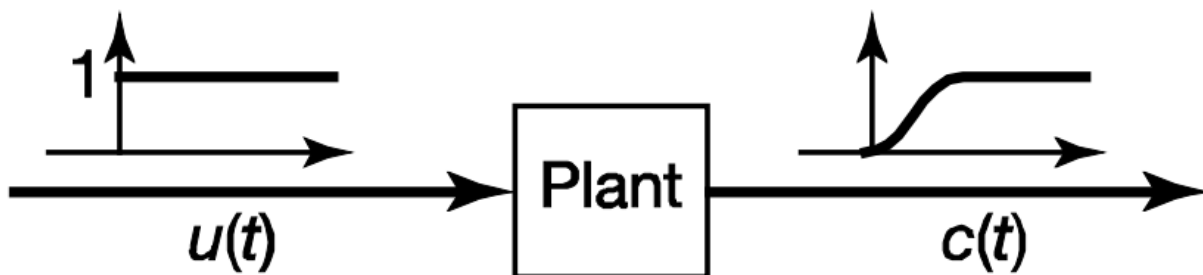
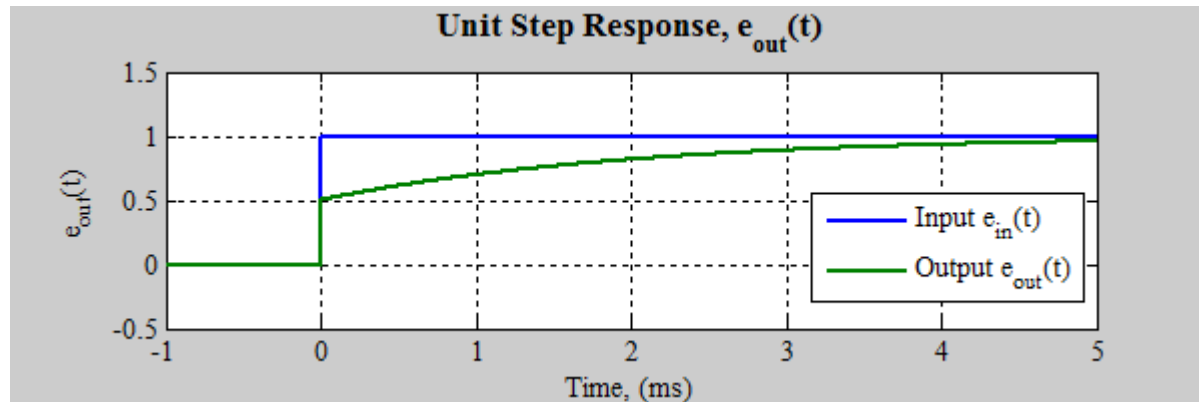
$$u(t) = \begin{cases} 1 & t \geq 0 \\ 0 & t < 0 \end{cases}$$

Which could be represented graphically as per the following:



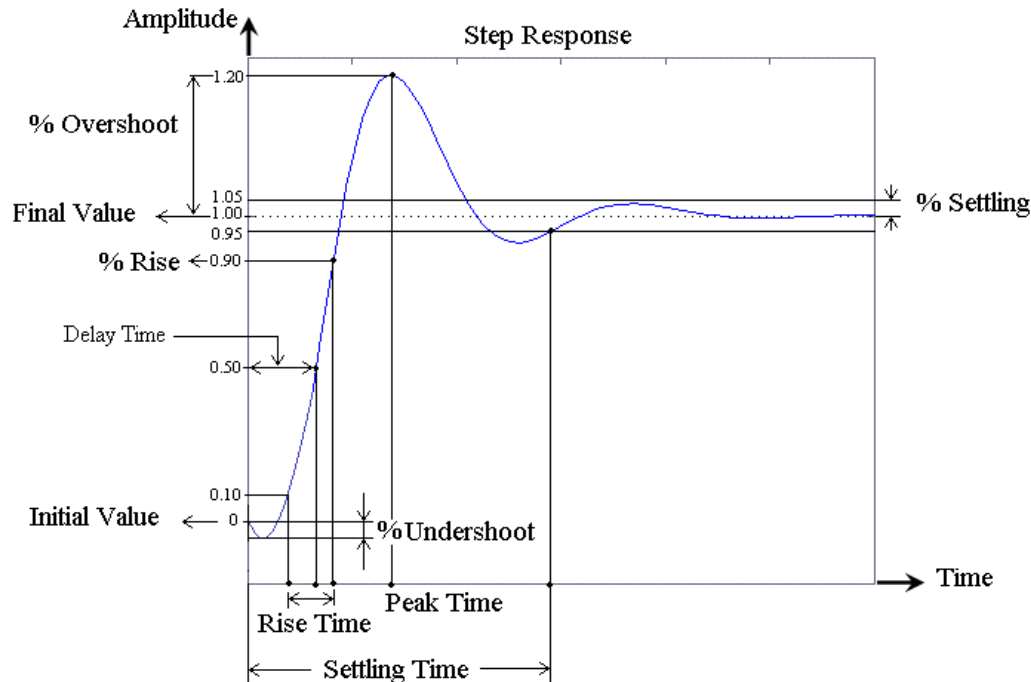
Any system has its own defined transfer function which is defined as the output per input. Each particular system or transfer function produces responses differently to an input of a unit step, this response is named “Step Response”.

For example, a plant of a DC Motor responds to a step input as the following:



As per illustrated in the previous graphs, the transfer function of the motor or any plant responded to the unit step input as graphed in the “Green Curve”. Then, this green curve is referred to as the “Step Response” of the plant. This could be imagined by a simple DC Motor, the motor takes some time to produce its maximum efficiency, which is named as **delay**, and also it may produce a spike rise in its torque as it just started, which is referred to as **overshoot**.

This step response could be used to analyze the transfer function of the system and deduce many characteristics about it and its stability. Any step response could be used to extract the following parameters.



- Delay Time (T_d):** The time by which the system reaches its 50% of the maximum value. It defines how the system response either in speed or slow manner.
- Settling Time (T_s):** The time required for the output to reach and steady within a given tolerance band.
- Rising Time (T_r):** The time by which the system reach from 5% to 95% or from 0% to 90%. It also gives an intuition about how the system will speedily respond and reach its maximum value.
- Percentage of Overshoot (%OS):** The maximum amount by which the response overshoots the steady-state value (the value at time goes to infinity) and is thus the amplitude of the first peak. The overshoot is often written as a percentage of the steady-state value.
- Peak Time (T_p):** The time required by response to reach its first peak. This parameter is sought to be decreased to achieve better stability.
- Steady State Value:** The value the system is to settle upon till the time goes to infinity.
- Tolerance Band:** a maximum allowable range in which the output can be settle. Generally, the tolerance bands are 2% or 5%.
- Initial Value:** the system value at time zero.
- Percentage of Undershoot:** The maximum amount by which the response undershoots its initial value and is thus the amplitude of the first peak under zero. The undershot is often written as a percentage of the initial value.

These parameters are named “**Transient Response Parameters**” which are the parameters of the step response of the system.

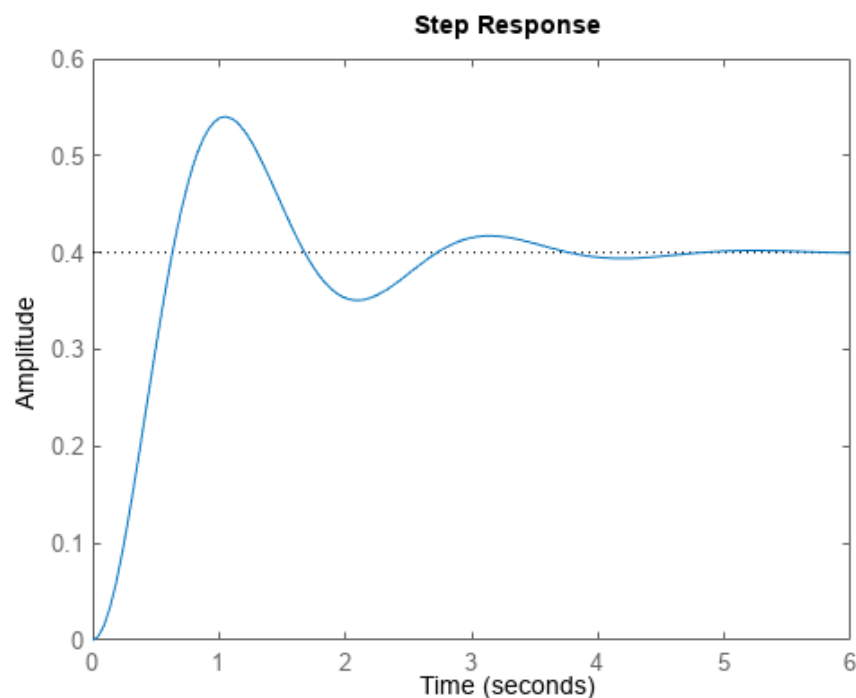
Example using MATLAB Control Systems Toolbox:

In this example a function named `step` is to be used. Step function plot the step response of a continuous-time system represented by the following transfer function which inputted to MATLAB using `tf` function as the next example.

Plot using MATLAB the step response of $G(s) = \frac{4}{s^2 + 2s + 10}$

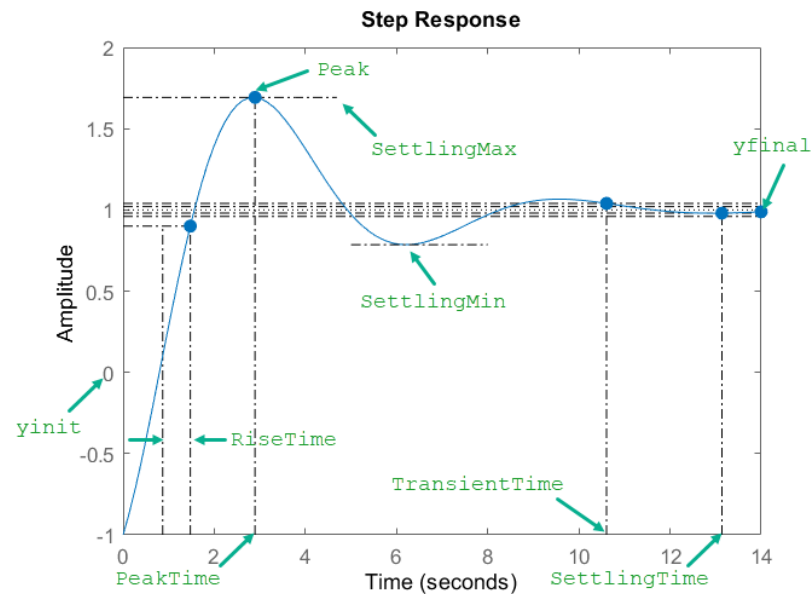
MATLAB Script:

```
sys = tf(4,[1 2 10]);  
step(sys)
```



NB: this could discretized using any sample to produce is twin in the k-domain.

The step plot automatically includes a dotted horizontal line indicating the steady-state response. In a MATLAB® figure window, you can right-click on the plot to view other step-response characteristics such as peak response and settling time as discussed before which are shown in the next figure:



It's good not that understanding this concept could be mapped for any step like input with any value. For example, powering up a DC motor with a 12V DC Battery is an example of an input of value 12 with a step input, only the amplitude of the step is not unity but 12. This is also could be applied in MATLAB and will be showed in the next example.

Another important parameter which could be evaluated from the transfer function of any system is the **Steady State Error** which is an error is defined as the **difference between the desired value and the actual value of a system output in the limit as time goes to infinity**. This steady state error could be found using MATLAB using a function named "sserror" in order to subtract the set point (SP) or the input from the final value of the transfer function, mathematically this is found using the final value theorem.

Example:

Use the previous transfer function with a step input of value 5 to get the steady state error of this function over this input.

Script:

```
sys = tf(4,[1 2 10]);
SP=5; %input value, if you put 1 then is the same as step(sys)
[y,t]=step(SP*sys); %get the response of the system to a step with amplitude SP
sserror=abs(SP-y(end)) %get the steady state error
```

This could easily get the steady state error in MATLAB, try the code.

Assignment: Verify the 3 steady state for all problems solved in the lecture using MATLAB.

In any **CLOSED LOOP** transfer function step response, a clear target for all control engineers is to:

1. Decrease the steady state error.
2. Decrease the percentage of overshoot.
3. Decrease the settling time.
4. Decrease the rise time.

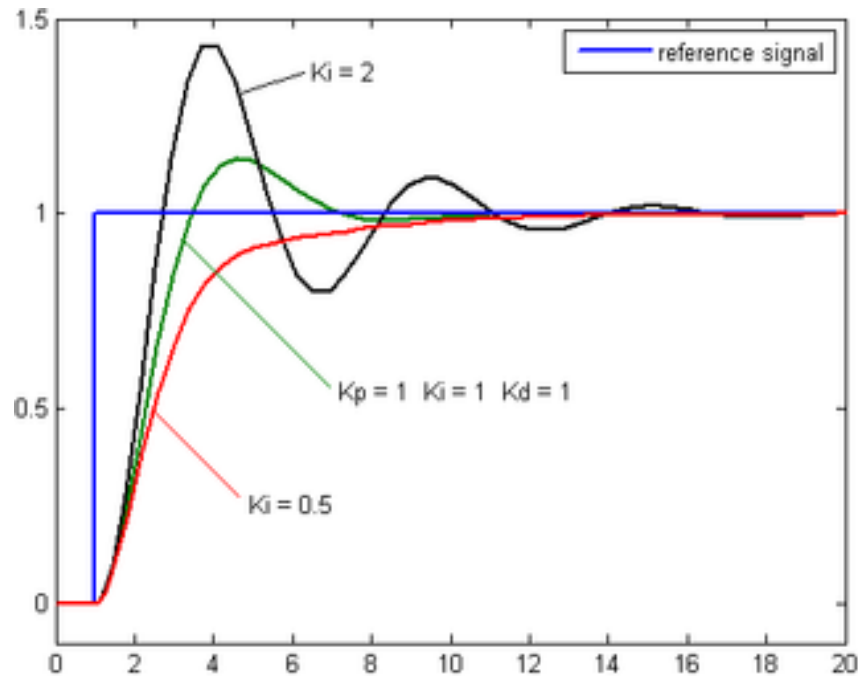
One of the most known and used control logic or digital controllers which was widely implemented in the analog domain and then it was transferred to the digital domain, is named “Proportional Integral Control” and its short form is the **PID Control**, which is characterized by three main parameters which are: the proportional gain, the integral gain, and the differential gain and are referred to by K_i , K_p , and K_d .

- A proportional controller (K_p) will have the effect of reducing the rise time, and will reduce but never eliminate the steady state error.
- An integral control (K_i) will have the effect of totally eliminating the steady state error but won't make the transient response worth it at all.
- An differential control (K_d) will have the effect of increasing the stability of the system, reducing the overshoot, and improving the transient response.

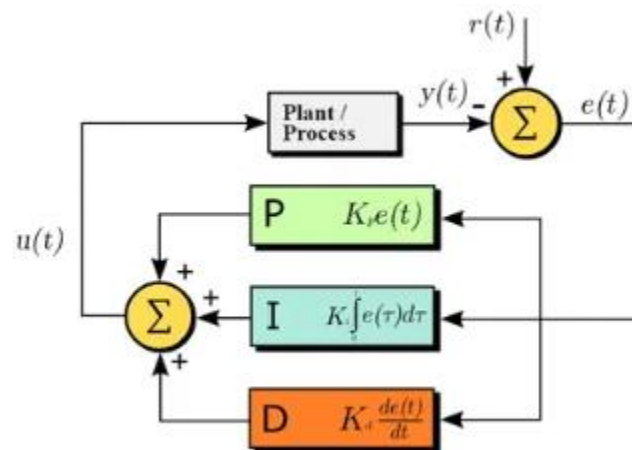
The effect of the three parameters in a CLOSED LOOP CONTROL SYSTEM is summarized in:

Closed Loop RESPONSE	RISE TIME	OVERSHOOT	SETTL TIME ING	S-S ERROR
K_p	Decrease	Increase	Small Change	Decrease
K_i	Decrease	Increase	Increase	Eliminate
K_d	Small Change	Decrease	Decrease	No Change

These THREE controllers effect could be visualized on the step response as per the following graph:



The block diagram visualization of the PID control could be graphed as the following:

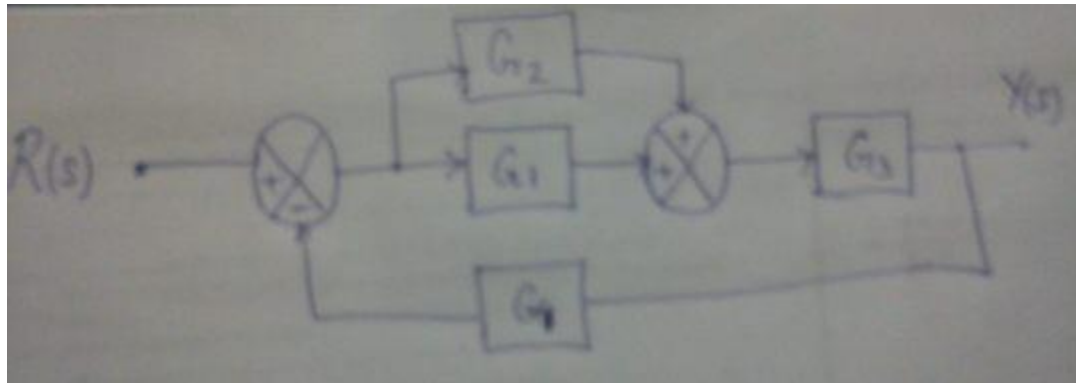


As discussed earlier, this could be reduced using the block diagram reduction rules of parallel connection which sums all of three function.

Block diagram reduction could be found using MATLAB as per the following example:

Example:

Solve the next problem using MATLAB block reduction ready functions:



Script:

```
num1 = [1 2];
den1 = [3 1 0];
G1 = tf(num1, den1)           %Making G1 as the tranfer function
G2 = tf( [2], [1 7] )
G3 = tf( [1 5], [1 6 3] )
G4 = tf( [1], [1 0] )

T1 = parallel(G1, G2)         %as G1 and G2 are in parallel
T2 = series(T1, G3)           %as T1 and G3 are in series
T = feedback(T2, G4, -1)      %as G4 is the negative feedback
```

Try the Script out :)

Assignment: Verify only 3 block diagrams from all problems solved in the lecture using MATLAB.

The mandate in the next scope is to optimize the three parameters in order to achieve the best PID gains to achieve the most near curve to the step response, which does mean to decrease the error. Its good visualize the difference between the perfect step input and the output as the error, which the controller (PID in this case) is seeking to decrease in order to best fir the step response shape. Also visualize what makes the input and output difference in shape?

Basically, it's the overshoot percentages, the delay time, and rising time. Optimizing all of them in the perfect fit, make the input and output as near as to each other also makes the error tends to be ZERO.

As a result, in the next phase, we will basically find a systematic method using the control systems to find the three PID gains which makes the maximum fitting to the input and make the error tends to be ZERO.

The derivation starts with calculating the manipulating signal which is the “Control Action” that is produced to eliminate the errors with all of its types in either steady state error or transient response errors which is $e_a(t)$, that could be calculated as the sum of the proportional error signal with derivative and integral of the error signal.

$$e_a(t) = e(t) + T_d \frac{de(t)}{dt} + K_i \int e(t) dt$$

In this scope of difference equations, it is recommended to solve it in the Laplace domain which produces the following equation:

$$E_a(s) = E(s) + sT_d E(s) + \frac{K_i}{s} E(s)$$

$$\text{or, } E_a(s) = E(s) \left[1 + sT_d + \frac{K_i}{s} \right]$$

- The T_d is the derivative time or the rate time which is a ratio between the K_d/K_p .
- The K_i is the integral gain which it is also a ratio between the K_p/T_i . T_i is the integral time which is the reset time
- The proportional gain which is K_p in this scope is set to be 1 but generally it shall be equal to K_p .

NB: in the last equation, it just taken the Error in S domain as common factor.

This equation could be then transferred to the Z-domain which is the “Digital Domain” by which all modern control systems work with.

$$E_a(Z) = M(Z) = K_p * E(Z) + \frac{K_d * (Z - 1)}{Z} E(Z) + \frac{K_i * Z}{Z - 1} E(Z)$$

In detail, K_p , K_i , and K_d could be evaluated from the following:

$$K_p = K - \frac{KT_s}{2T_i}$$

$$K_i = \frac{KT_s}{T_i}$$

$$K_d = \frac{KT_d}{T_s}$$

*K is the analog proportional gain.

* T_s is the sampling time.

Then, the Impulse Transfer Function for the PID controller could be obtained the following:

$$G(Z)_{pid} = \frac{M(Z)}{E(Z)} = K_p + \frac{K_i * Z}{Z - 1} + \frac{K_d * (Z - 1)}{Z}$$

Another view of the PID controllers -in analog domain-, is to visualize this effect by adding a ZERO or/and a POLE to the system transfer function of the plant. To note, the controllers implements could be only PI controller which is defined as “ADDING POLE” effect and in the same manner for the PD controllers implemented are defined as “ADDING ZERO” effect which is seen in speeding up the response in the forward direction; which give us a definition of the PID compensator in the S-domain as the following:

$$G(s)_{pid} = K * \frac{s + z}{s + p}$$

*This PID transfer function in the s-domain the one beyond the previous proof of PID on the z-domain.

NB: in some occasions, the PI controller is named “Lag Compensator” which compensates the error to reach the input by lagging some parameters and the PD controller is named “Lead Compensator” which compensates the error to reach the input by leading, means by speeding up the transfer function.

Practically speaking, the MATLAB PID Tuner is considered one of the most effective tools to find the three gains which are the proportional, integral, and derivative gain which could be known from the **pdtune** function and then transfer this transfer function to the discrete domain using **c2d** function, which are used in the following example:

Example:

The transfer function of a DC motor is made using system identification tool and is found to be:

$$G(s) = \frac{286}{s^2 + 6s + 14.6}$$

Find the PID controller parameters and total system TF in Z domain at a sampling time of 0.01 seconds.

Script:

```
sys = tf ([286],[1,6,14.6])
```

then,

`sys_original =`

`286`

`-----`

`s^2 + 6 s + 14.6`

`>> pid_test= pidtune(sys_original,'Pid')`

`pid_test =`

`1`

`Kp + Ki * --- + Kd * s`
`s`

`with Kp = 0.119, Ki = 0.312, Kd = 0.0102`

`Continuous-time PID controller in parallel form.`

Then, use the `c2d` function at 0.01 seconds on the `pidtune`.

Then, cascade this output with the `c2d` of the motor transfer function given in the problem to reach the final output.

NB: The 'pid' model could be replaced with 'pi' model or 'pd' model.

PID Tuning could be more effectively made using SIMULINK tool in MATLAB as per the following example:

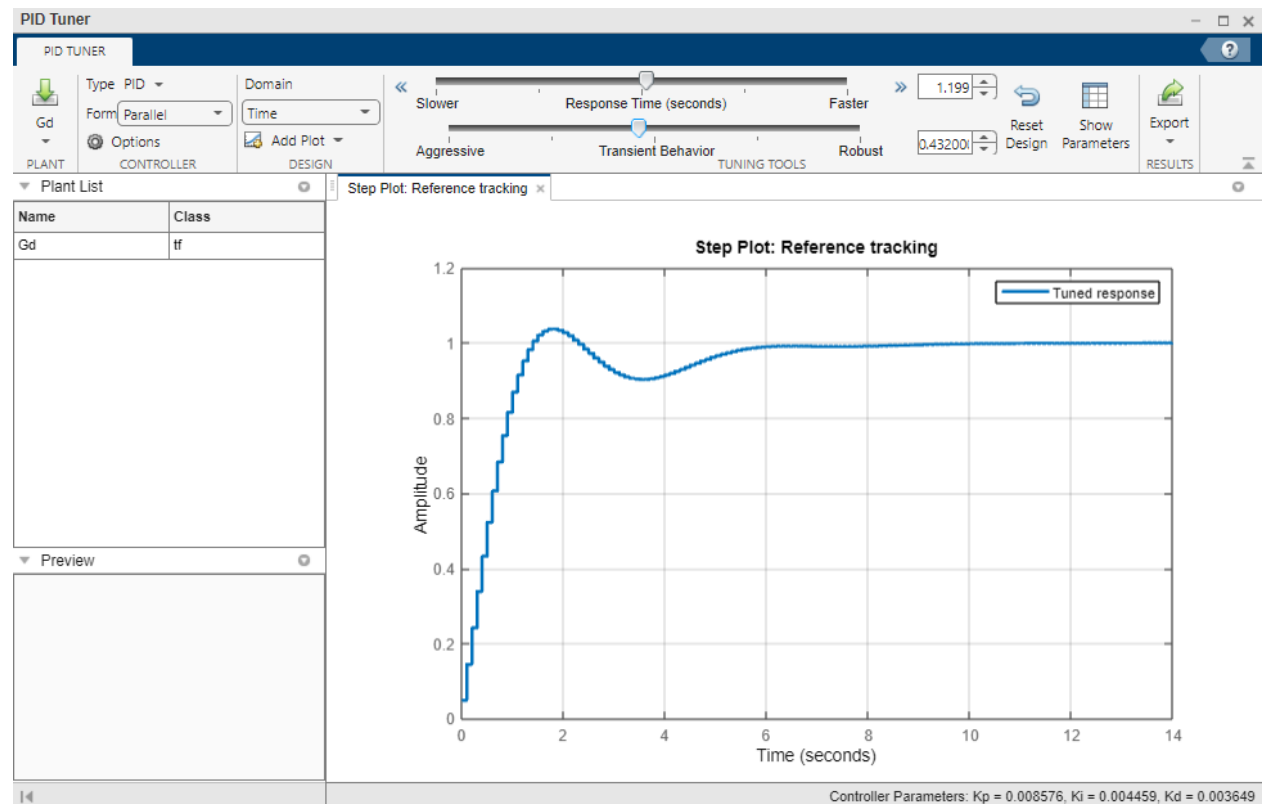
Example:

`Gc = tf ([286],[1, 1, 1]);`

`Gd = c2d(Gc,0.1);` % Create discrete-time plant

`pidTuner(Gd,'pid')` % Launch PID Tuner

Then, the PID tuner screen is to appear and could easily optimized by changing the transient response and the response time behavior to obtain the three parameters of the PID Controller at the screen bottom.



Assignment: The transfer function of the SG90 servo motor with 5v step input is found as the following:

$$\frac{-0.8054 s^2 + 4204 s + 1.444e06}{s^3 + 469.1 s^2 + 3.676e04 s + 1.356e06}$$

Find the PID controller parameters and the total system transfer function. Then, plot the 5v unit step input for the servo motor alone and with it the PID controller on it in the same graph with different legends with naming them.

HINT: This transfer function is to be used in your project calculations.

HINT: Use hold on, grid on, legend, and title function to perform this assignment well.

BONUS: Use the system identification tool to find the transfer function of a given system. The input and output data could be found [here](#).

BONUS: Use the PUD Tuner to find the servo motor main PID parameters.

In our project, the transfer function to be controlled using the PID control is the SERVO motor transfer function which was illustrated above. In the next phase, the mandate it to implement

the digital PID controller found above on Arduino in order to produce the suitable manipulating signal on the SERVO motor plant, this done in the next steps:

1. Get the three gain values from the MATLAB PID tuning and define them.
2. Get the error in the first cycle which is the set point or the input - the output measured by a certain sensor.
3. Get the PID Value which could be found from the in the coming equations:
4. Sum them to get the total PID Value.

$$Error = Input - Output$$

$$P_{value} = Error * K_p$$

$$I_{value} = Accumative\ Error * K_i$$

$$D_{value} = Differenial\ Error * K_d$$

*The accumulative error is the error from all previous cycles.

*The differential error is the error difference from the last two cycles.

5. Map this PID value to the actuator which is to derive the system for the next cycle.

Sample Code:

```
#define kp 0 %this value shall be set from the MATLAB pid tuner  
#define ki 0 %this value shall be set from the MATLAB pid tuner  
#define kd 0 %this value shall be set from the MATLAB pid tuner
```

```
double priError = 0;
```

```
double toError = 0;
```

```
}
```

```
%Write needed viod set-up lines according to the system
```

```
void loop() {
```

```
    PID();
```

```
}
```

```
void PID() {
```

```
int measured = measuredValue; %suppose there is a measured value

int setPoint = 15; %set manually

double error = setPoint - dis; %Calculate the error which is difference between the measured
value and the set point the use assigned

double Pvalue = error * kp; %calculate the P value

double Ivalue = toError * ki; %caculate the I value

double Dvalue = (error - priError) * kd; %calculate the D value

double PIDvalue = Pvalue + Ivalue + Dvalue; %Sum of to get the PID controller

priError = error; %get the value calculated from the first loop then put in the next loop

toError += error; %get the value by increment the priError and += is the compound addition
which does mean that toError = error + toError

Serial.println(PIDvalue); %print the PID value

}
```

Assignment: write the a functional Arduino code to implement the PID controller for a servo motor taking the input as a variable named “distance” and produces an output of angle based on PWM.

Root Locus:

The Root locus is the locus of the roots of the characteristic equation by varying system gain K from zero to infinity, this in CLOSED LOOP systems.

*All points on the root locus shall satisfy the conditions.

It's used as a method allows us to determine the traces of the poles in the s-plane as any one coefficient of the closed-loop system (for example the controller gain) is varied from least to highest number.

Root locus graph could be found by realizing what's common between all the system poles and try to translate all of these equations into root locus representation rules.

The characteristic equation of the closed loop control system is

$$1 + G(s) N(s) = 0$$

The previous equation could be written as:

$$G(s) H(s) = K \frac{D(s)}{N(s)}$$

- K represents the multiplying factor.
- N(s) represents the numerator term having (factored) Nth order polynomial of 's'.
- D(s) represents the denominator term having (factored) Mth order polynomial of 's'

Then, substitute in the first equation:

$$1 + K \frac{D(s)}{N(s)} = 0$$

Then,

$$D(s) + K N(s) = 0$$

This could be divided in two cases which are:

1- $K = 0$

That means, the closed loop poles are equal to open loop poles when K is zero.

2- $K = \infty$

If $K = \infty$, then $N(s) = 0$. It means the closed loop poles are equal to the open loop zeros when K is infinity.

From above two cases, we can conclude that the root locus branches start at open loop poles and end at open loop zeros.

In addition, the points on the root locus shall satisfy the angle condition. So, the angle condition is used to know whether the point exist on root locus branch or not. The value of K could be found for these points on the root locus branches by using magnitude condition. So, we can use the magnitude condition for the points, and this satisfies the angle condition.

Characteristic equation of closed loop control system is

$$1 + G(s) N(s) = 0$$

Then,

$$G(s) N(s) = -1 + 0j$$

Then, the phase angle of the previous is:

$$\angle G(s) N(s) = (2n + 1)\pi$$

The angle condition is the point at which the angle of the open loop transfer function is an odd multiple of 180 degrees.

In addition, the magnitude condition is that the point (which satisfied the angle condition) at which the magnitude of the open loop transfer function is one.

$$|G(s) N(s)| = 1$$

By applying the previous magnitude and angle conditions, the traces of the poles could be represented on the s complex domain. This method clarifies all the system poles which could give an indication, where the poles should be shifted for further stability.

The root locus could be graphed using MATLAB by using rlocus() function in the control systems tool box.

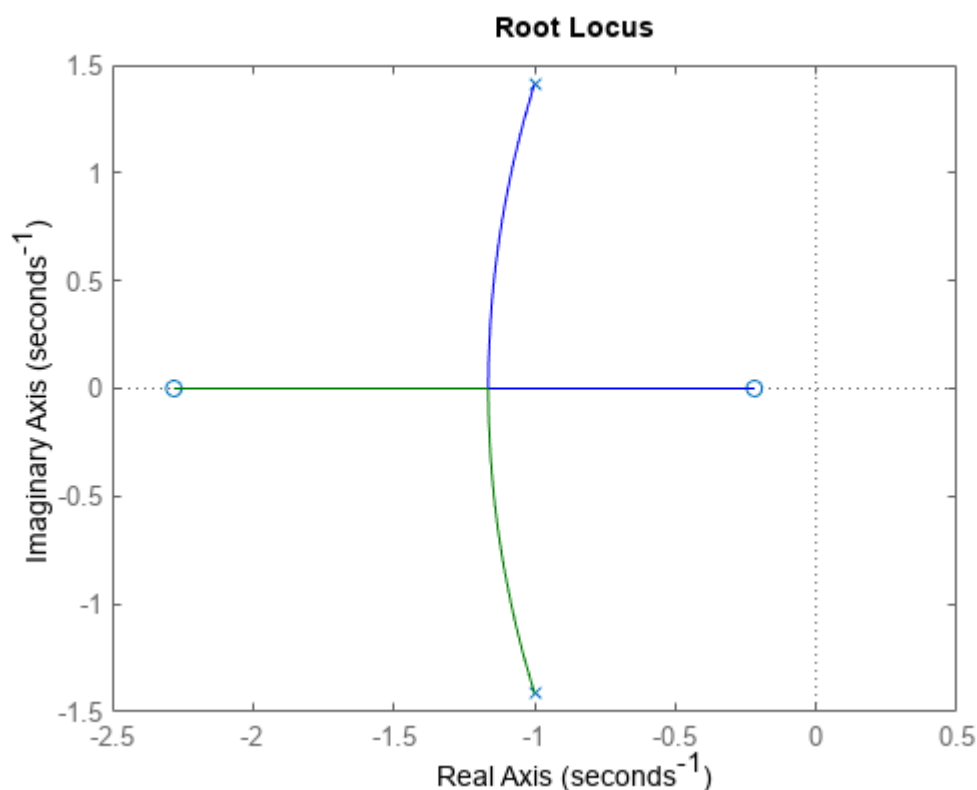
Example:

Plot the root locus of the following transfer function using MATLAB:

$$G(s) = \frac{2s^2+5s+1}{s^2+2s+1}$$

Solution:

```
sys = tf([2 5 1],[1 2 3]);  
rlocus(sys)
```



The poles of the system are denoted by x , while the zeros are denoted by o on the root locus plot. You can use the menu within the generated root locus plot to add grid lines, zoom in or out, and also invoke the Property Editor to customize the plot.

Example: Plotting Multiple Root Loci on MATLAB:

After defining more than ONE transfer function in the s-domain. The SISO model shall be called to plot many root loci in one plot as the following:

```
load('sisoModels.mat','sys1','sys2','sys3');
```

Create the root locus plot using `rlocus` and specify the color for each system. Also add a legend to the root locus plot, using the following:

```
rlocus(sys1,'b',sys2,'k',sys3,'r')
```

```
hold on
```

```
legend('sys1','sys2','sys3')
```

```
hold off
```

*a, b, c are the colors of the root loci graph.

*legend is used to identify each root locus by naming it.

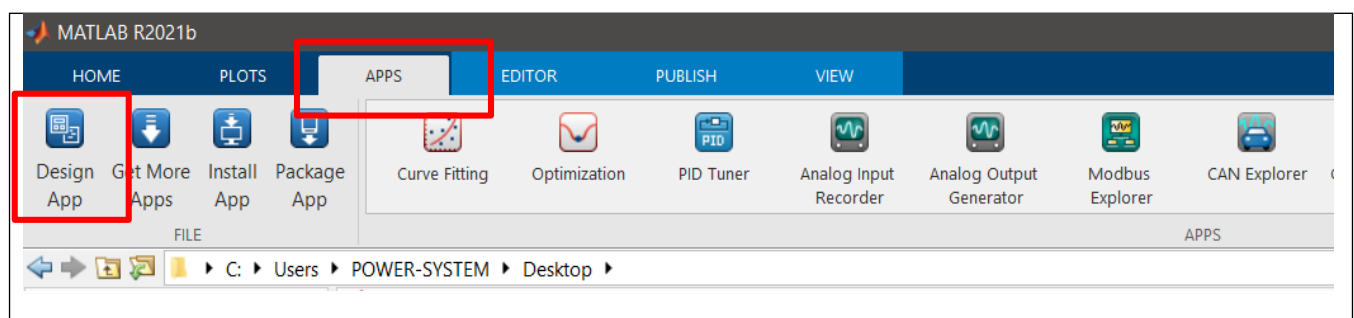
*Hold on and off are used to retain plots in the current axes so that new plots added to the axes do not delete existing plots.

Assignment: verify using MATLAB Control Box `rlocus` function THREE root locus problems solved in the lecture.

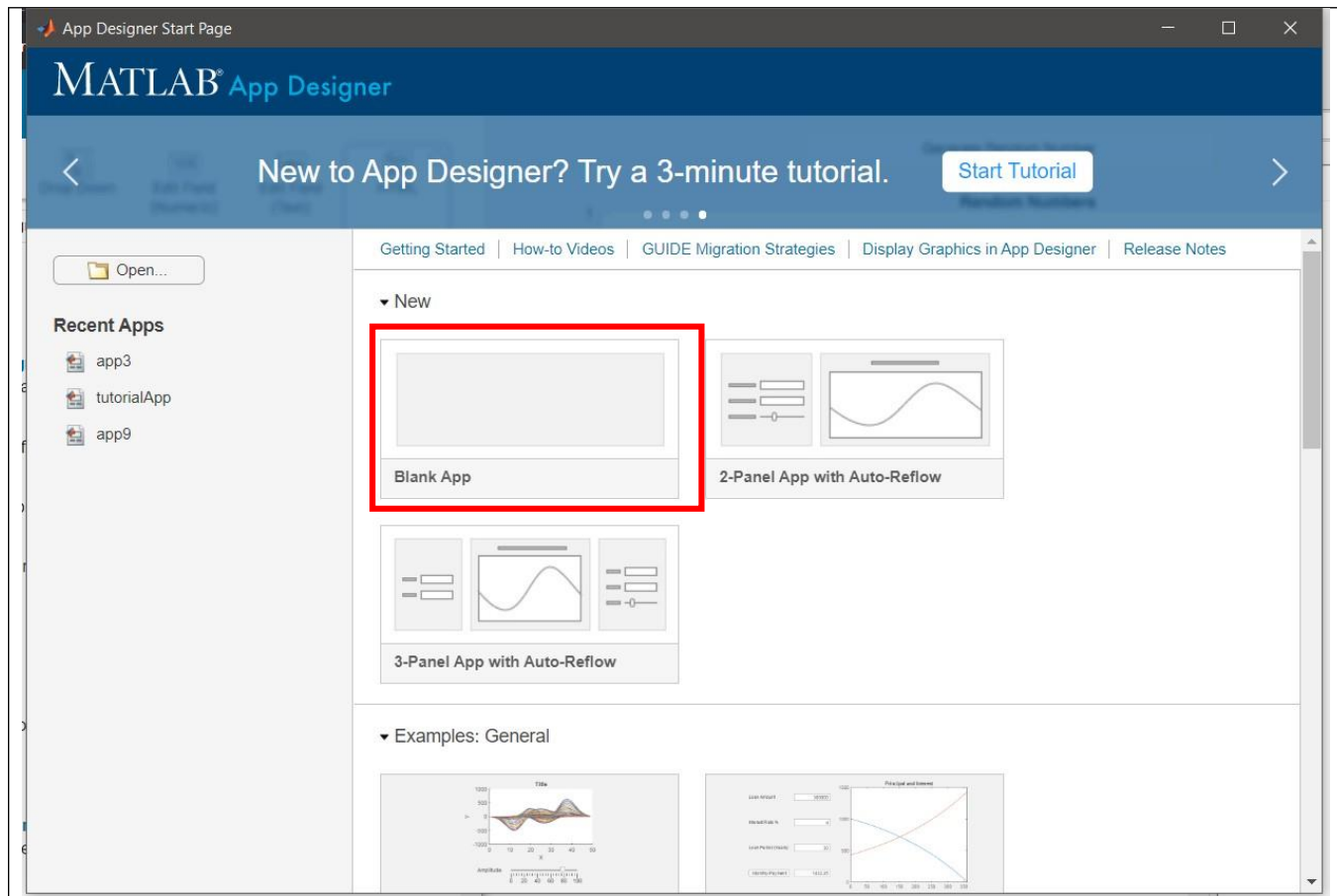
Extra: Making a GUI and connecting it to Arduino

Making a sample GUI plotting the Z poles and step response could be done as per the following:

1. Open MATLAB App designer “Design App”



2. Open Blank App

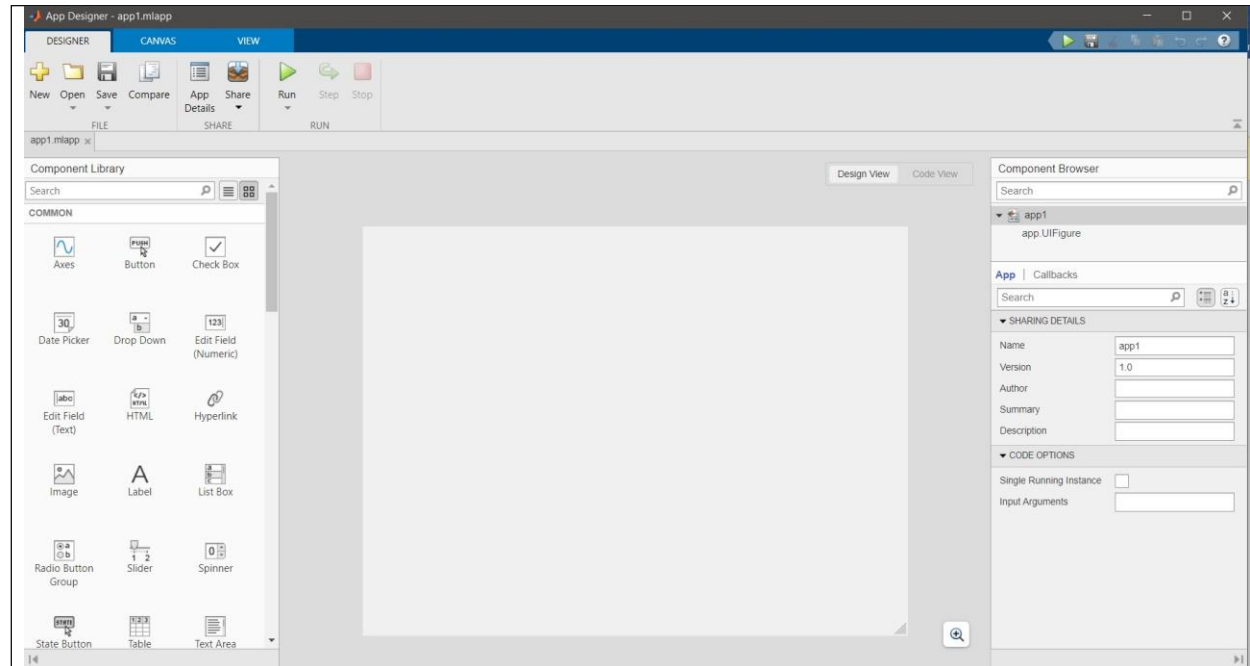


Fall 2022

Modern Control Systems

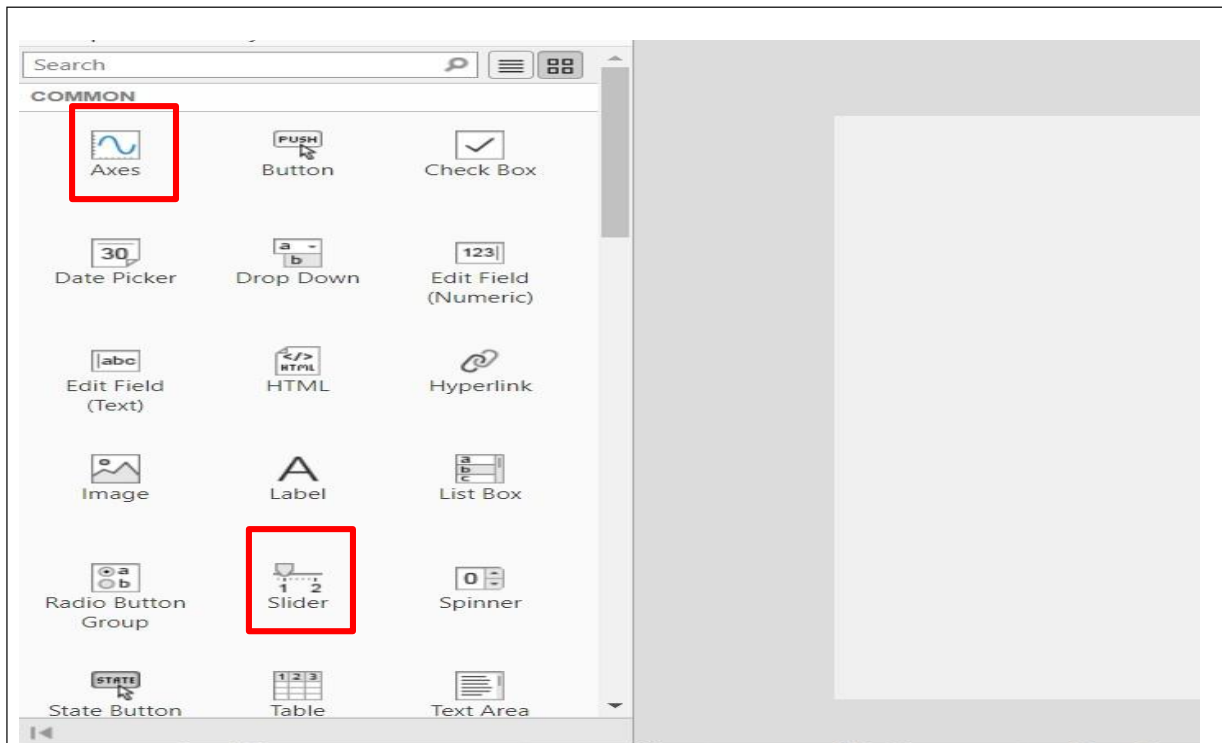
Intelligent Systems Program

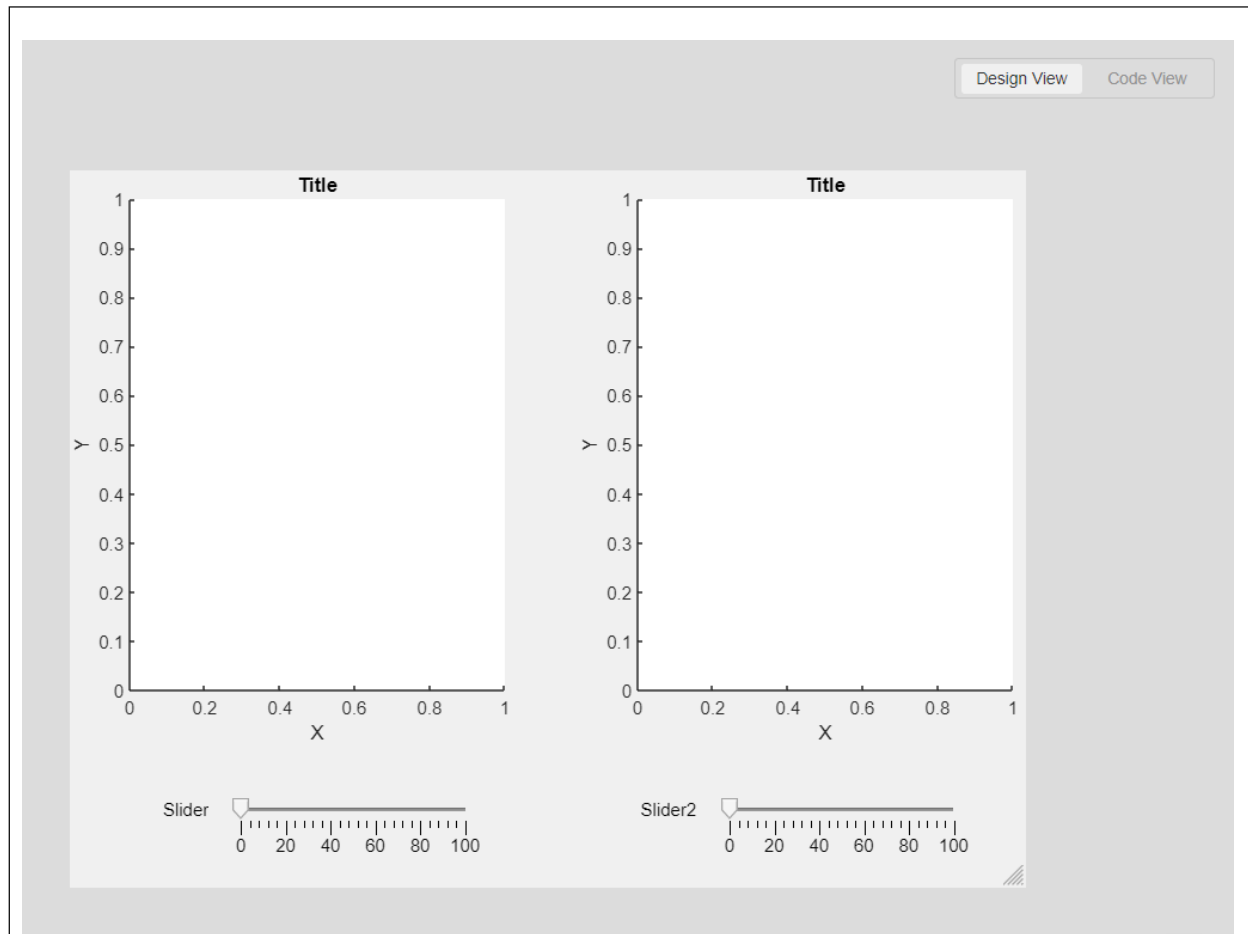
Dr. Mahmoud Gamal



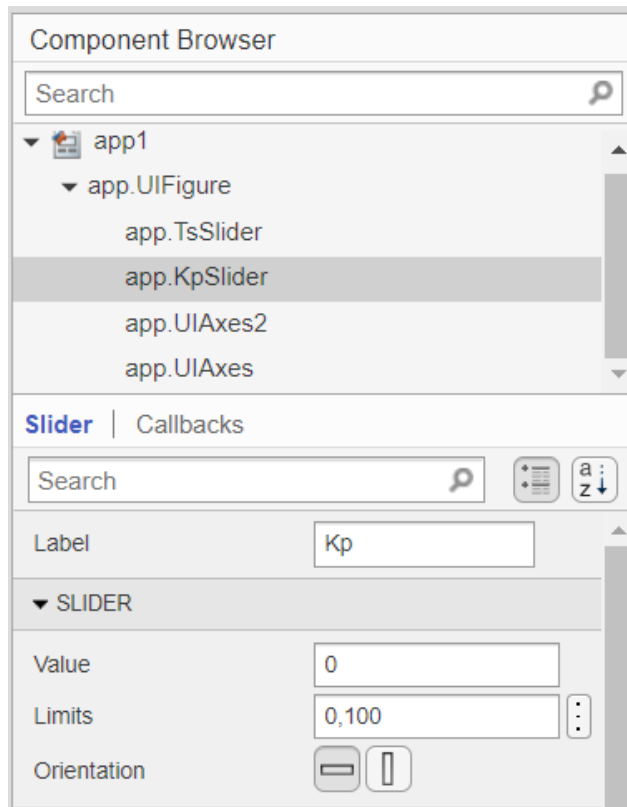
3. Pull and drop

- 2 Axes
- 2 Sliders

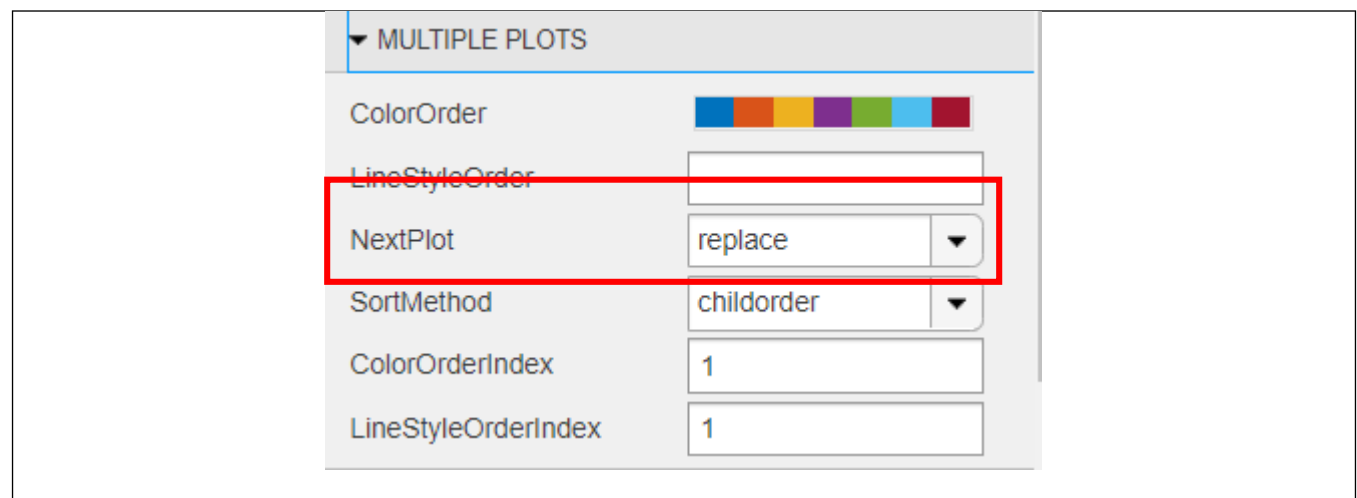




4. Click on slider and rename both sliders and set their limits from component browser



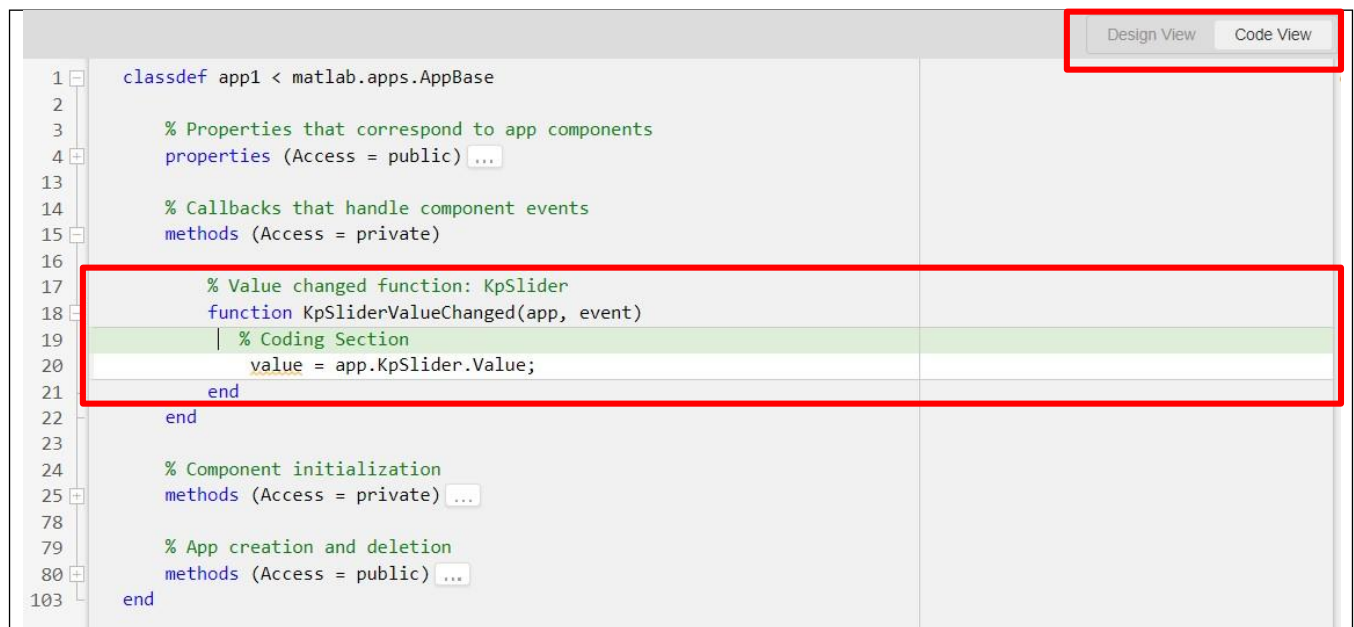
5. Click on each Axes block and from component browser.



6. From component browser Right click on “app.KpSlider”

- Callbacks then Add ValueChangedFcn callback

7. Open Code View



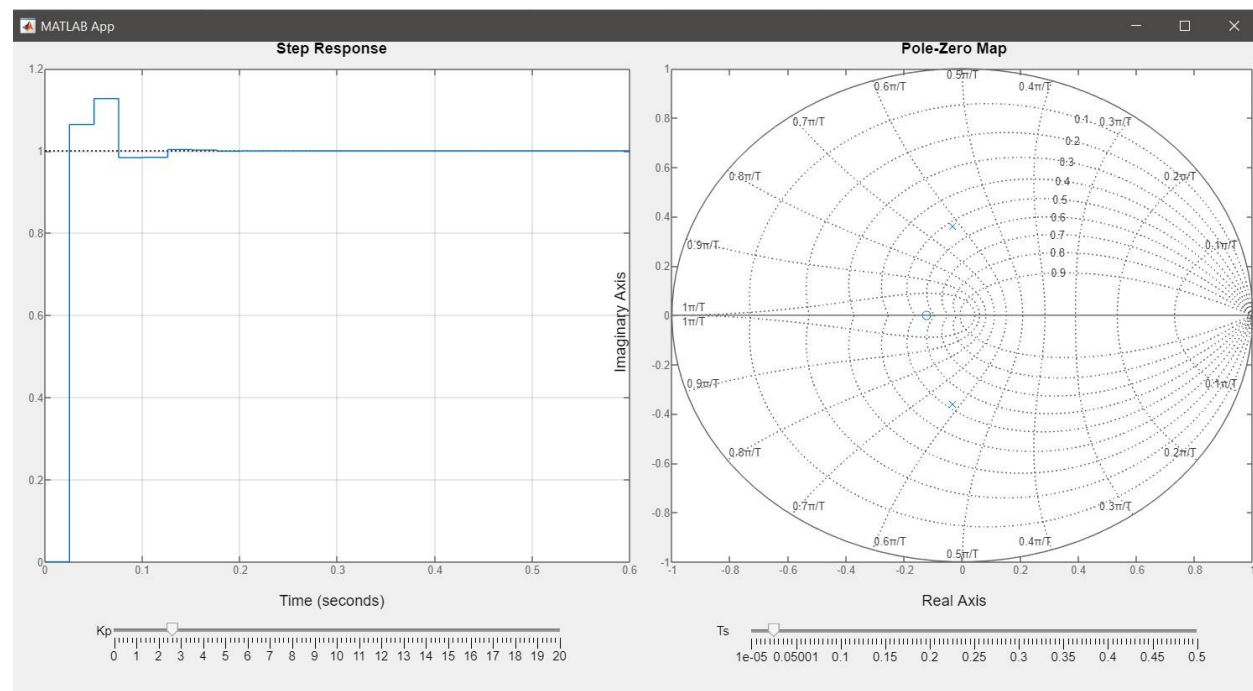
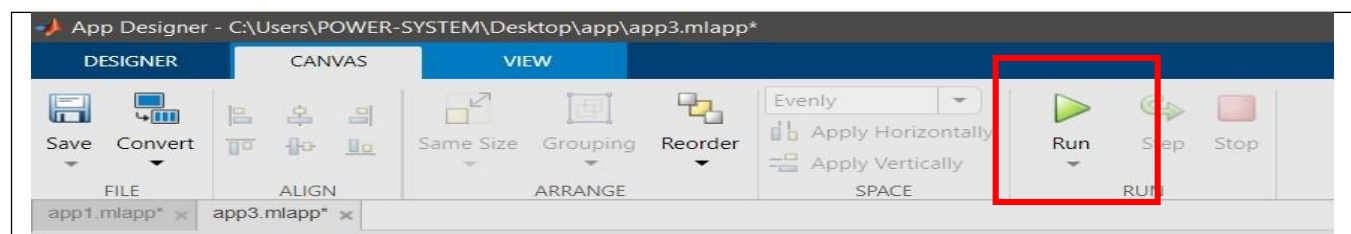
The image shows the MATLAB App Designer interface with the 'Code View' tab selected. The code is for a class named 'app1' that inherits from 'matlab.apps.AppBase'. It includes sections for properties, callbacks, component initialization, and app creation/deletion. A red box highlights the 'ValueChangedFcn' callback function 'KpSliderValueChanged', which contains a coding section that updates the 'value' property of 'app.KpSlider'.

```
1 classdef app1 < matlab.apps.AppBase
2
3     % Properties that correspond to app components
4     properties (Access = public) ...
13
14     % Callbacks that handle component events
15     methods (Access = private)
16
17         % Value changed function: KpSlider
18         function KpSliderValueChanged(app, event)
19             % Coding Section
20             value = app.KpSlider.Value;
21         end
22     end
23
24     % Component initialization
25     methods (Access = private) ...
78
79     % App creation and deletion
80     methods (Access = public) ...
103 end
```

8. For GUI Code

```
% Callback function: KpSlider, KpSlider
function KpSliderValueChanged(app, event)
    % Read Slider Value
    Kp_value = app.KpSlider.Value;
    Ts = app.TsSlider.Value;
    % Insert G(s) Transfer function
    % Transform G(s) --> G(z) Transfer function
    % Plot Step response for the system
    step(app.UIAxes,sys)
    % Plot poles and zeros for the system
    % zgrid(app.UIAxes2,'on')
    % pzmap(app.UIAxes2, sys)
end
```

9. Click Run



Note: first set the sampling time then change the Kp value.

In addition, Set the sampling time then change the Kp value using GUI and Write Your comments for each state after Plotting Poles on z-map.

Connecting MATLAB to Arduino could be as the following:

MATLAB® and Simulink® address several challenges with traditional Arduino programming. The products support two primary workflows:

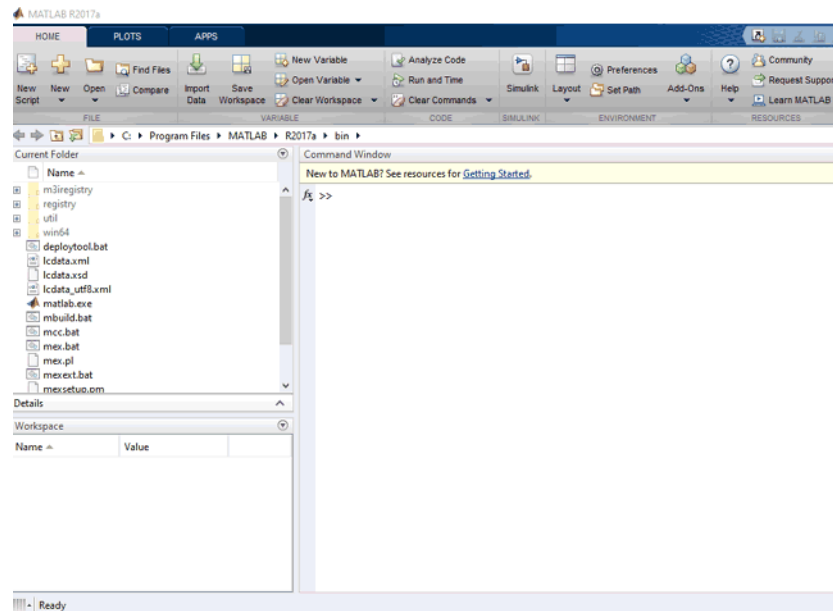
- Read, write, and analyze data from Arduino sensors.
- Develop algorithms that run standalone on the Arduino device.

Benefits of using Simulink for Arduino programming:

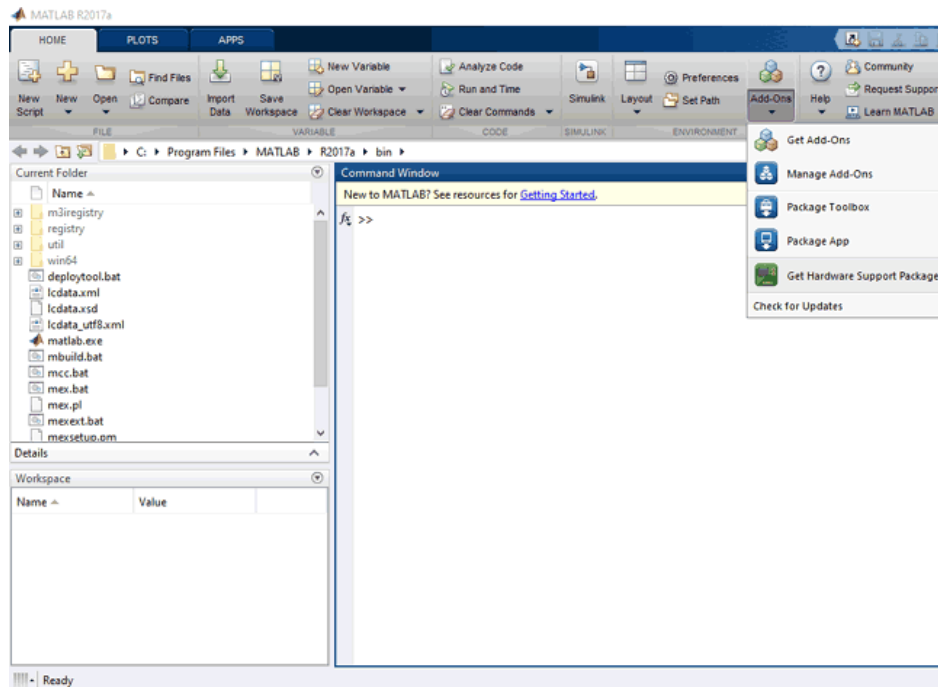
- Develop and simulate your algorithms in Simulink and use automatic code generation to run them on the device
- Incorporate signal processing, control design, state logic, and other advanced math and engineering routines in your hardware projects
- Interactively tune and optimize parameters in Simulink as your algorithm runs on your Arduino
- Easily modify algorithms to run on other low-cost and commercial hardware platforms

Setup Hardware Support Package for MATLAB:

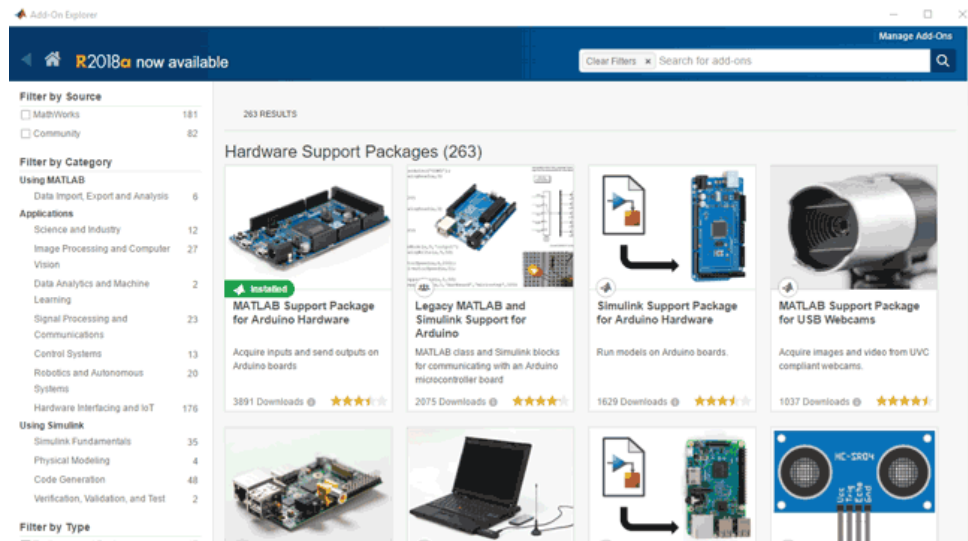
- Step 1. Start MATLAB (latest Version preferred).



Step 2. In the Environment section, select Add-Ons > Get HardwareSupport Packages.



Step 3. It will start the Add-On explorer window.



Step 4. Click on MATLAB Support Package for Arduino Hardware.

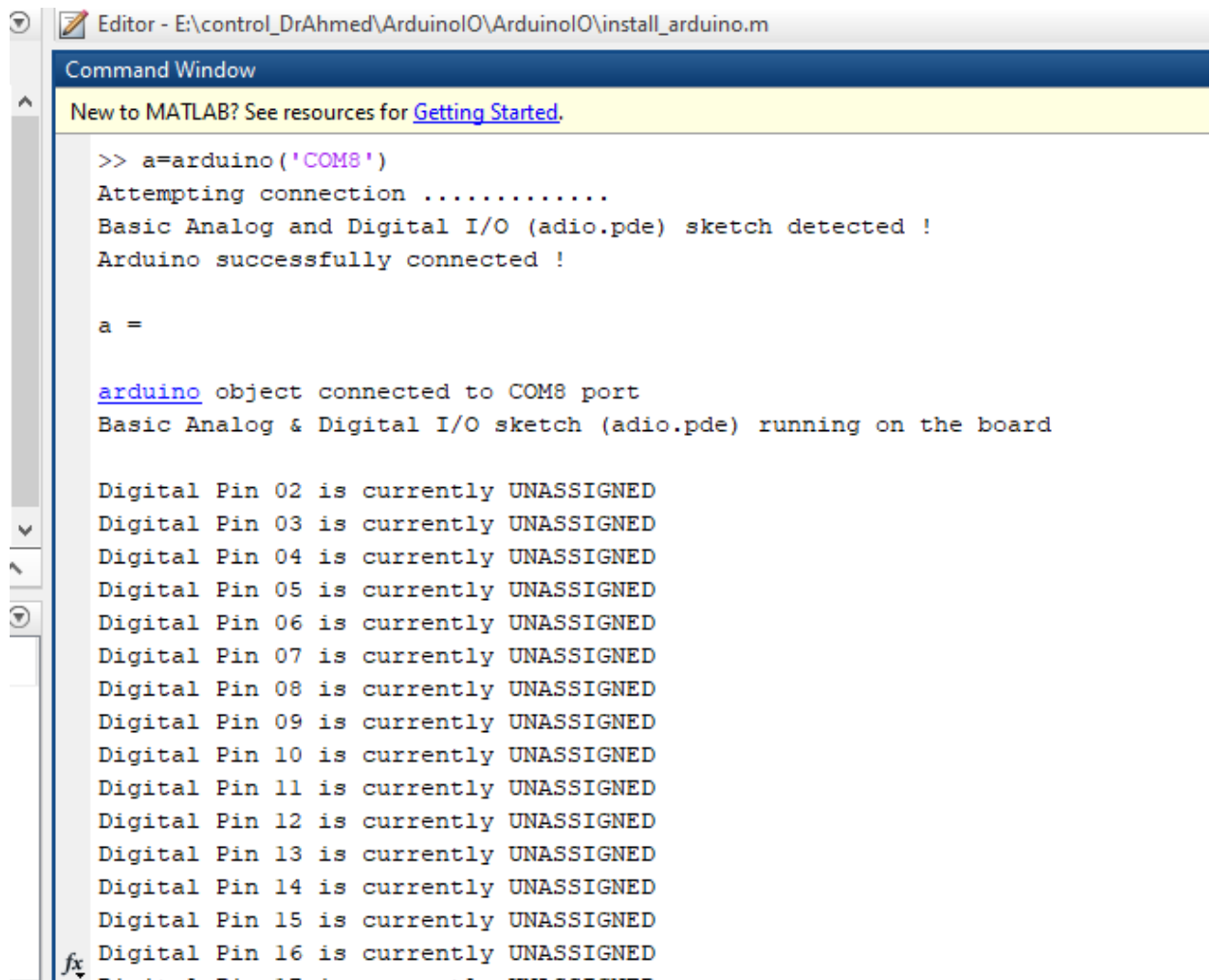
Step 5. Click on Install, now the installer will ask you to log into your MathWorks account. If you don't have MathWorks account, you can create an account during installation.

Step 6. After logging in, Accept the license agreement and proceed to installation.

Step 7. Now, wait for the package to download and install.

Step 8. Now you have successfully installed Arduino Support Package for MATLAB.

Step 9. verify Arduino software package using digitalWrite() function to light built-in LED on Arduino and MATLAB command window .



```
Editor - E:\control_DrAhmed\ArduinoIO\ArduinoIO\install_arduino.m

Command Window

New to MATLAB? See resources for Getting Started.

>> a=arduino('COM8')
Attempting connection .....
Basic Analog and Digital I/O (adio.pde) sketch detected !
Arduino successfully connected !

a =

arduino object connected to COM8 port
Basic Analog & Digital I/O sketch (adio.pde) running on the board

Digital Pin 02 is currently UNASSIGNED
Digital Pin 03 is currently UNASSIGNED
Digital Pin 04 is currently UNASSIGNED
Digital Pin 05 is currently UNASSIGNED
Digital Pin 06 is currently UNASSIGNED
Digital Pin 07 is currently UNASSIGNED
Digital Pin 08 is currently UNASSIGNED
Digital Pin 09 is currently UNASSIGNED
Digital Pin 10 is currently UNASSIGNED
Digital Pin 11 is currently UNASSIGNED
Digital Pin 12 is currently UNASSIGNED
Digital Pin 13 is currently UNASSIGNED
Digital Pin 14 is currently UNASSIGNED
Digital Pin 15 is currently UNASSIGNED
Digital Pin 16 is currently UNASSIGNED
```

```
>> a.pinMode(13, 'OUTPUT')
>> a.digitalWrite(13,0)
>> a.digitalWrite(13,1)
>> clear a
```

Step 10. verify Arduino software package for Simulink using a digitaloutput to light a LED

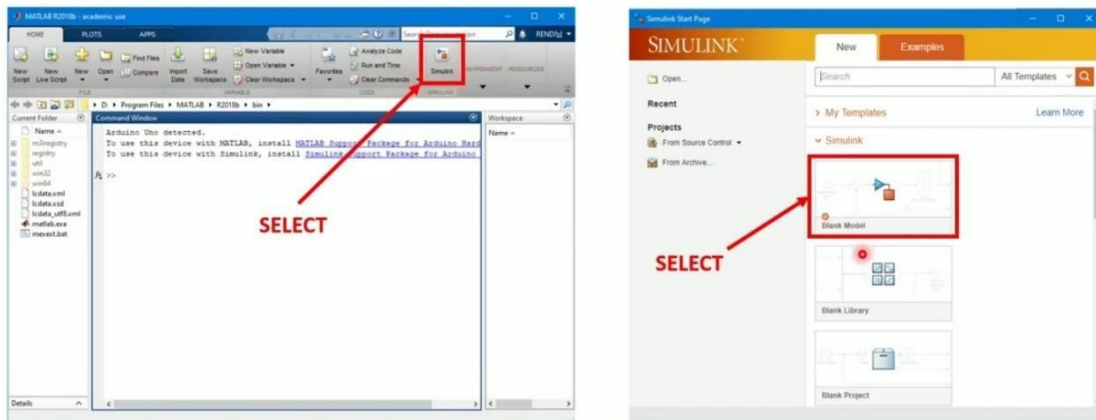
Simulink Setup:

To ensure that your computer can properly communicate with the Arduino board, build, and run the following Simulink diagram using the steps below:



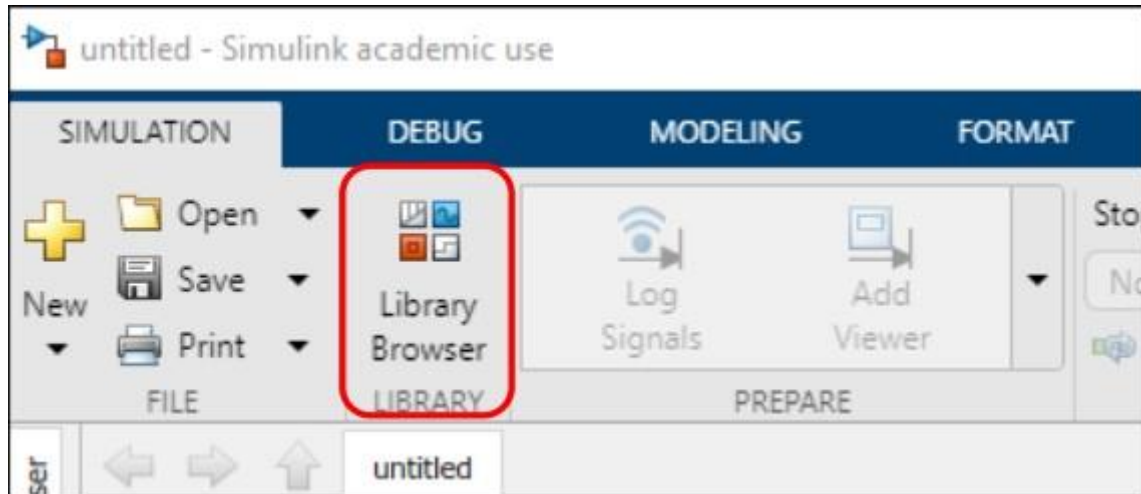
Open MATLAB then open a new Simulink model.

Simulink Start Page will appear. Click in Blank Model option to create a new model



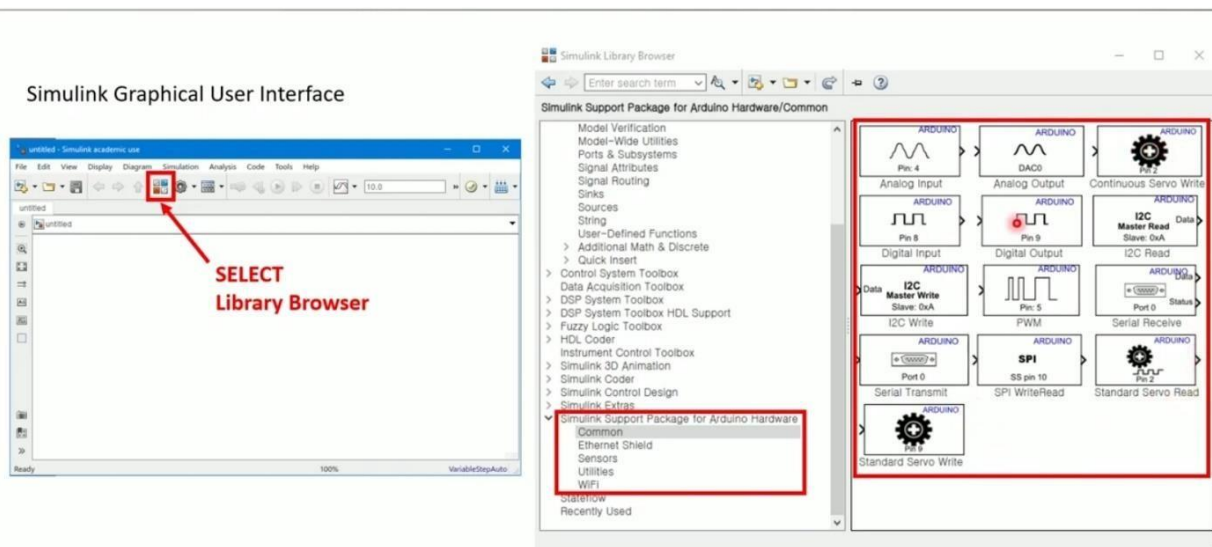
Find and assemble the required blocks.

- Pulse Generator is under View->Library Browser->Simulink->Sources



Digital Output is under View->Library Browser->Simulink Support Package for Arduino Hardware View->Library Browser->Simulink Support Package for Arduino Hardware->Common)

Copy Digital Output block in your Simulink module.



Change the output pin to 13, which is connected to the onboard LED. This is done by double clicking on the Digital Output block.

Double click on the Pulse Generator to change the amplitude to 1, the Period to 10, and the pulse width to 50%.

Fall 2022
Intelligent Systems Program
Modern Control Systems
Dr. Mahmoud Gamal



Under the tools menu go to 'Run on Target Hardware' and click 'Prepare to Run'.
In the window that pops up you will need to select Arduino uno in the drop down
for Target Hardware.

At this point the on-board LED connected to pin 13 should start blinking on and
off.